# Thread Scaling in Shared Memory Matrix Multiplication

*Assignment 3 - Universidad de Las Palmas de Gran Canaria*

*Student: Ivan Giacomo Colella*

## Abstract

This study investigates how thread count affects dense matrix multiplication in Java. Equivalent sequential and row-partitioned parallel implementations use the same i-k-j algorithm, double precision arrays and seeded inputs. Four matrix sizes and four parallel thread counts are evaluated in three independent JVM processes per configuration, with 20 warm-ups and 10 measured repetitions per process. Eight threads give the lowest aggregated runtime at every tested size. At n = 1024, runtime decreases from 760.943 ms to 229.886 ms, giving a speedup of 3.31 and efficiency of 41.4%. Scaling is sublinear overall and includes substantial variability in some configurations. A speedup of 2.40 with two threads at n = 512 is examined alongside the faster parallel one-thread baseline; its cause remains unresolved.

## 1 Introduction and objective

The research question is: how does thread count affect runtime and process memory for an equivalent shared-memory matrix multiplication implementation? The changed factors are matrix side length n and parallel worker count p. The hypothesis is that distributing independent output rows reduces runtime, while task management and shared hardware resources limit efficiency. This hypothesis is evaluated only for the tested machine and size range.

## 2 Background and implementation

For square matrices, C[i,j] is the sum of A[i,k] × B[k,j] over k. Both implementations perform cubic arithmetic work using the i-k-j loop order and flattened row-major double[] storage. The inner j loop accesses consecutive elements of B and C. The assignment requires an equivalent sequential baseline and the parallel one-thread case [1].

MatrixMultiplication.multiplySequential computes all rows directly. ParallelMatrixMultiplication uses a fixed ExecutorService with p workers. Worker w processes rows from floor(w × n / p) inclusive to floor((w + 1) × n / p) exclusive. These intervals cover all rows without overlap. Inputs are read-only, and each output element has one writer. invokeAll followed by Future.get waits for every task and propagates failures.

No explicit SIMD extension is evaluated. JVM automatic optimisations are not measured separately, so no vectorisation benefit is claimed. No numerical library or BLAS implementation is called; p describes pool workers, not all JVM service threads.

# 3 Experimental methodology

| Item | Configuration |
|---|---|
| **Runtime** | Temurin OpenJDK and javac 21.0.12.1; Windows 11 build 26300 |
| **Hardware** | Intel Core i7-10510U @ 1.80 GHz; 4 physical cores; 8 logical processors; 15.83 GiB RAM |
| **Input** | n = 128, 256, 512, 1024; java.util.Random; seeds 2026 and 2027 |
| **Values and layout** | Uniform generator values in [−1, 1); double; flattened row-major arrays |
| **Methods** | Sequential with p = 1; parallel with p = 1, 2, 4, 8 |
| **Repetitions** | 3 JVM forks × 10 measurements; 20 warm-ups per fork |
| **Execution controls** | Randomised order seed 20261107; 2 s pause between processes |
| **Budgets** | 120 s per process; 1024 MiB sampled RSS; JVM heap maximum 512 MiB |
| **Monitoring** | Python 3.13.5; psutil 7.2.2; requested RSS sampling interval 10 ms |

All methods use identical logical inputs for a given n. The largest worker count equals the number of logical processors, so worker count does not exceed available hardware execution contexts. The CPU model was subsequently identified using Get-CimInstance Win32_Processor. The suite does not set CPU affinity; the effective inherited affinity was not recorded. The operating system schedules the workers, so exact core placement is not controlled.

System.nanoTime measures each multiplication call. The timed region includes argument checks, allocation and zero-initialisation of C, and arithmetic. For the parallel method it also includes task construction, submission and waiting. Input generation, reference calculation, correctness checking and console/CSV output are excluded. The pool is constructed before warm-up and reused; pool shutdown is excluded. Initial worker creation occurs during warm-up.

Small correctness tests in CorrectnessTests.java cover known products, identity and zero matrices, repeated calls, uneven row partitions, n = 1, 2, 3, 7, 16, 31 and p = 1, 2, 4, 8. Random cases use an independent i-j-k reference. For benchmark runs, every result is compared elementwise to the sequential result using $|\text{actual} - \text{expected}| \leq 10^{-10} +$

$10^{-10}$|expected|, rejecting non-finite outputs. This allows small floating-point discrepancies while checking all elements. All 600 recorded measurements have verified=true; benchmark verification alone is not an independent test of the sequential algorithm.

Each fork is summarised by the median of its 10 timed measurements. The reported configuration runtime is the median of its three fork medians. Variability is described by the minimum and maximum fork median and their sample coefficient of variation (CV). Speedup S = Tseq / Tp and efficiency E = S / p use these aggregated times. No measurements are excluded.

Memory is the sampled peak RSS over the entire Java process lifetime, summarised by its median across forks. It includes inputs, reference and result arrays, JVM memory, warm-ups and workers. It is not a measure of kernel-only memory or Java heap alone, and sampling can miss brief peaks. All 60 processes completed successfully; n = 1024 was both the largest attempted and largest successful size, not an established capacity limit.

# 4 Results

| n | Method | p | Time ms | Fork range ms | CV % | S | E % |
|---|---|---|---|---|---|---|---|
| **128** | Seq | 1 | 1.181 | 1.148–1.356 | 9.1 | 1.00 | 100.0 |
| **128** | Par | 1 | 1.219 | 1.205–1.377 | 7.6 | 0.97 | 96.9 |
| **128** | Par | 2 | 0.725 | 0.677–0.760 | 5.8 | 1.63 | 81.5 |
| **128** | Par | 4 | 0.715 | 0.682–0.725 | 3.2 | 1.65 | 41.3 |
| **128** | Par | 8 | 0.589 | 0.573–0.652 | 7.0 | 2.01 | 25.1 |
| **256** | Seq | 1 | 9.199 | 9.174–9.735 | 3.4 | 1.00 | 100.0 |
| **256** | Par | 1 | 9.809 | 9.761–9.819 | 0.3 | 0.94 | 93.8 |
| **256** | Par | 2 | 4.831 | 4.780–5.621 | 9.3 | 1.90 | 95.2 |
| **256** | Par | 4 | 4.215 | 2.608–6.435 | 43.5 | 2.18 | 54.6 |
| **256** | Par | 8 | 3.528 | 3.353–4.891 | 21.5 | 2.61 | 32.6 |
| **512** | Seq | 1 | 81.184 | 73.840–89.240 | 9.5 | 1.00 | 100.0 |
| **512** | Par | 1 | 68.628 | 67.418–69.407 | 1.5 | 1.18 | 118.3 |
| **512** | Par | 2 | 33.841 | 33.664–35.895 | 3.6 | 2.40 | 120.0 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **512** | Par | 4 | 29.353 | 28.733–29.369 | 1.2 | 2.77 | 69.1 |
| **512** | Par | 8 | 19.195 | 18.671–19.880 | 3.2 | 4.23 | 52.9 |
| **1024** | Seq | 1 | 760.943 | 760.366–780.002 | 1.5 | 1.00 | 100.0 |
| **1024** | Par | 1 | 732.346 | 730.852–749.550 | 1.4 | 1.04 | 103.9 |
| **1024** | Par | 2 | 498.336 | 494.999–545.779 | 5.5 | 1.53 | 76.3 |
| **1024** | Par | 4 | 326.899 | 316.366–340.295 | 3.7 | 2.33 | 58.2 |
| **1024** | Par | 8 | 229.886 | 220.964–266.386 | 10.1 | 3.31 | 41.4 |

**Table 1. Runtime and scaling for all 20 configurations. Each row represents 30 measurements across three forks; ranges and CV refer to the three fork medians, not individual samples.**

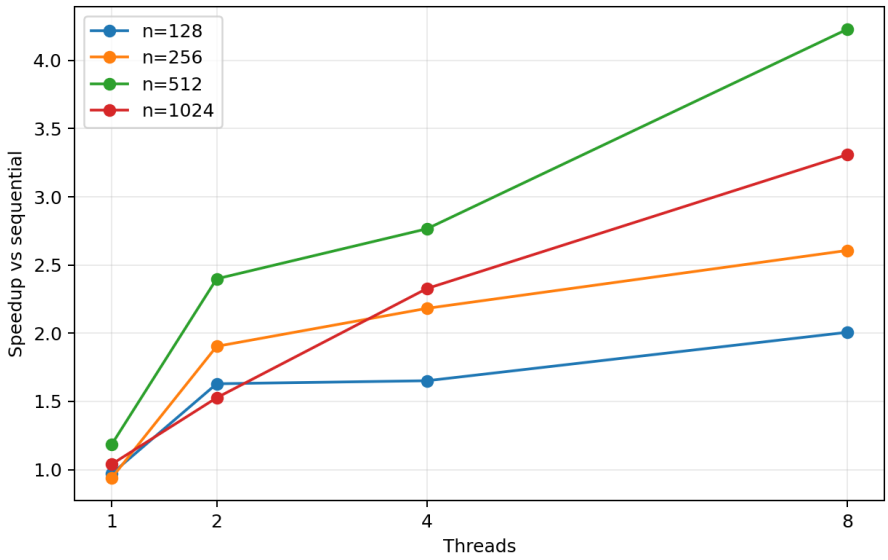



**Figure 1. Speedup relative to the equivalent sequential implementation. The highest speedup occurs with eight threads at every tested size. Lines connect measured configurations and do not imply results for other counts.**

# 5 Memory and discussion

| n | Sequential MiB | Par 1 MiB | Par 2 MiB | Par 4 MiB | Par 8 MiB |
|---|---|---|---|---|---|
| **128** | 47.80 | 45.58 | 45.94 | 47.38 | 46.55 |
| **256** | 56.75 | 58.08 | 57.51 | 58.56 | 58.77 |
| **512** | 62.34 | 62.52 | 63.07 | 64.29 | 64.97 |
| **1024** | 76.17 | 77.12 | 77.62 | 78.87 | 81.95 |

**Table 2. Median of sampled process peak RSS across three forks. At n = 1024, sequential RSS is 76.17 MiB and eight-thread RSS is 81.95 MiB. Small non-monotonic changes at other sizes cannot be**

**interpreted as precise allocation differences because RSS includes the JVM and its process-wide behaviour.**

Parallel execution improves runtime with two or more workers in every aggregated comparison. With eight threads, efficiency ranges from 25.1% at n = 128 to 52.9% at n = 512. At n = 1024 the speedup is 3.31, below the 4.23 observed at n = 512. Therefore a larger input does not guarantee better scaling. Task overhead, memory access, scheduling and CPU frequency are plausible contributors, but no hardware counters or frequency measurements were collected to distinguish them.

The parallel one-thread case reveals an important baseline difference. At n = 128 and 256 it is approximately 3.2% and 6.6% slower than sequential. At n = 512 it is faster: 68.628 ms versus 81.184 ms. Dividing the parallel one-thread time by the two-thread time gives 2.03, whereas the required sequential baseline yields 2.40. Thus the apparent superlinear result includes a pre-existing difference between the two code paths. The source has equivalent arithmetic and fair timing boundaries, but distinct method structure and execution threads can receive different JVM optimisations. This remains a hypothesis rather than a demonstrated cause.

The n = 256, p = 4 configuration has fork medians from 2.608 to 6.435 ms and a CV of 43.5%; p = 8 at the same size has a CV of 21.5%. These observations are retained. The aggregate ranking is descriptive of this study and should not be treated as a stable prediction for another run. Only three independent JVM forks were collected, and the shortest measurements are below one millisecond. More forks, longer timed batches for small sizes, recorded effective affinity and optional JVM compilation or GC diagnostics would strengthen a follow-up investigation.

# 6 Conclusions

Within this experiment, row-parallel multiplication reduces runtime and eight workers give the fastest aggregated result at every size. The benefit is substantial but not proportional to worker count: n = 1024 achieves 3.31× speedup at eight workers. The parallel one-thread baseline and fork variability are essential to interpreting these results. The study does not establish universal optimal thread counts or identify the cause of the unusual speedup at n = 512.

## AI assistance disclosure

OpenAI ChatGPT supported experimental planning, code development and debugging. The measurements were produced by running the programs locally. AI-generated text was not used as experimental evidence.

# 7 Reproducibility materials

GitHub repository: https://github.com/ivanc25/Assignment_03_BD

The benchmark source is identified by recorded commit b046427df6482a86800532c3440e99338f4953bf. The benchmark environment recorded results/ as untracked. Analysis code was added afterwards. Retain the final source, scripts, plan.json, environment.json, progress.json and all per-process measurements.csv, resources.json, memory_samples.csv and java.log files.

Run the following PowerShell commands from the repository root, using Java 21 and the recorded Python dependencies. The suite requires a new output directory, so use a new name when repeating the experiment. Analysis requires matplotlib 3.11.2; the original suite environment also recorded NumPy 2.5.4, while the later analysis console used NumPy 2.5.3. The analysis script does not import NumPy directly.

```
New-Item -ItemType Directory -Force out

javac -d out src/*.java

java -cp out CorrectnessTests

python scripts/run_suite.py results/final_v2

python scripts/analyze_results.py results/final_v2 results/analysis_final_v2
```

The original study is stored in results/final_v1 and its analysis in results/analysis_final_v1. Input files are unnecessary for regeneration: MatrixInputs.java defines the generator, dimensions, range and seeds. Independent exported reference checks can be produced with ExportValidation.java; their execution is not evidenced by the supplied benchmark logs.

# References

[1] Big Data course, Universidad de Las Palmas de Gran Canaria. Individual Assignments Matrix Multiplication, academic year 2026–2027, revised proposal. Common protocol, Assignment 3 and submission requirements. Supplied course file BD_Individual_Assignments_2026_2027.pdf.